

Python Assignment

1. Create set and add new items from user.

```
s = set()
n = int(input("How many items? "))
for _ in range(n):
    s.add(input("Enter item: "))
print("Set:", s)
```

2. Delete an item given by user and print the set.

```
s = {"apple", "banana", "mango"}
item = input("Enter item to delete: ")
s.discard(item)
print("Updated Set:", s)
```

3. Create student info set and marks set, display whole record.

```
student = {"Ankit", "101", "BCA", "A"}
marks = {85, 90, 88}
print("Student Record:", student, marks)
```

4. Create a list taking 5 input numbers from user.

```
nums = []
for i in range(5):
    nums.append(int(input("Enter number: ")))
print(nums)
```

5. Replace marks of existing student record list.

```
marks = [70, 80, 90]
index = int(input("Enter index to replace: "))
new_mark = int(input("Enter new mark: "))
marks[index] = new_mark
print(marks)
```

6. Create two employee record lists.

```
emp1 = ["Rahul", 101, "9876543210"]
emp2 = ["Amit", 102, "9123456780"]
print(emp1)
print(emp2)
```

7. Create salary list and display employee salary records.

```
names = ["Rahul", "Amit"]
salaries = [25000, 30000]
for i in range(len(names)):
    print(names[i], "-", salaries[i])
```

8. Add 5 student records dynamically.

```
students = []
for i in range(5):
    name = input("Name: ")
    cls = input("Class: ")
    roll = input("Roll No: ")
    marks = input("Marks: ")
    students.append([name, cls, roll, marks])
print(students)
```

9. Delete a record.

```
records = ["A", "B", "C"]
item = input("Enter record to delete: ")
records.remove(item)
print(records)
```

10. Create a tuple with values inserted by user.

```
values = []
n = int(input("How many values? "))
for _ in range(n):
    values.append(input("Enter value: "))
t = tuple(values)
print(t)
```

11. Remove an item from tuple.

```
t = (1, 2, 3, 4)
lst = list(t)
lst.remove(int(input("Enter item to remove: ")))
t = tuple(lst)
print(t)
```

12. Print employee details through inheritance.

```
class Employee:
    def __init__(self, name, emp_id, dept, salary):
        self.name = name
        self.emp_id = emp_id
        self.dept = dept
        self.salary = salary

class Project(Employee):
    def __init__(self, name, emp_id, dept, salary,
                  pname, members, start):
        super().__init__(name, emp_id, dept, salary)
        self.pname = pname
        self.members = members
        self.start = start

obj = Project("Rahul", 101, "IT", 50000,
              "ERP", 5, "01-01-2026")
print(obj.__dict__)
```

13. Show student record using multiple inheritance.

```
class Student:
    def student_info(self):
        self.name = "Ankit"
        self.roll = 12

class Section:
    def section_info(self):
        self.section = "A"

class Marks(Student, Section):
    def marks_info(self):
        self.marks = 89
        print(self.name, self.roll,
              self.section, self.marks)

obj = Marks()
obj.student_info()
obj.section_info()
obj.marks_info()
```

14. Show user details using abstraction and encapsulation.

```
class User:
    def __init__(self, name, email, password,
                  contact, cv):
        self.name = name
        self.email = email
        self.__password = password
        self.contact = contact
        self.cv = cv

    def display(self):
        print(self.name, self.email,
              self.contact, self.cv)

u = User("Aman", "aman@gmail.com",
         "1234", "9999999999", "cv.pdf")
u.display()
```

15. Program for function overloading.

```
class Demo:
    def add(self, a, b=0, c=0):
        print(a + b + c)

d = Demo()
d.add(5)
d.add(5, 10)
d.add(5, 10, 15)
```

16. Show base class method if same method exists in subclass.

```
class Parent:
    def show(self):
        print("Parent Method")

class Child(Parent):
    def show(self):
        Parent.show(self)

obj = Child()
obj.show()
```

17. Print area of square using lambda function.

```
side = int(input("Enter side: "))
area = lambda s: s * s
print("Area =", area(side))
```

18. Print factorial using lambda function.

```
from functools import reduce

n = int(input("Enter number: "))

fact = lambda n: reduce(
    lambda x, y: x * y,
    range(1, n + 1)
)

print("Factorial =", fact(n))
```

19. Generate area based on user choice.

```
choice = input(
    "Enter shape (square/rectangle/circle): "
)

if choice == "square":
    s = float(input("Side: "))
    print("Area =", s * s)

elif choice == "rectangle":
    l = float(input("Length: "))
    b = float(input("Breadth: "))
    print("Area =", l * b)

elif choice == "circle":
    r = float(input("Radius: "))
    print("Area =", 3.14 * r * r)
```